

Assignment No 10

Name: Sciddhanto Sinha

Class: CSE, SY-2(B)

Roll No.:2213111

Enrollment No.: MITU21BTCS0553

Aim: (Functions)

Write a program to ATM machine operation utilizing classes, inheritance concepts for following operations (Assume customer has savings bank account with respective bank:

- a. Check Balance
- b. Withdraw Amount
- c. Deposit Amount
- d. Mini Statement
- e. Change Pin

Theory:

Inheritance is the capability of one class to derive or inherit the properties from another class.

Python Inheritance Syntax

Class BaseClass:

{Body}

Class DerivedClass(BaseClass):

{Body}

Creating a Parent Class

Creating a Person class with Display methods.

A Python program to demonstrate inheritance

```
class Person(object):
```

```
# Constructor
```

```
def __init__(self, name, id):
```

```
    self.name = name
```

```
    self.id = id
```

```
# To check if this person is an employee
```

```
def Display(self):
```

```
    print(self.name, self.id)
```

```
# Driver code
```

```
emp = Person("Satyam", 102) # An Object of Person
```

```
emp.Display()
```

Output:

Satyam 102

Creating a Child Class

Here Emp is another class which is going to inherit the properties of the Person class(base class).

```
class Emp(Person):
```

```
    def Print(self):
```

```
        print("Emp class called")
```

```
Emp_details = Emp("Mayank", 103)
```

```
# calling parent class function
```

```
Emp_details.Display()
```

```
# Calling child class function
```

```
Emp_details.Print()
```

Output:

Mayank 103

Emp class called

Example of Inheritance in Python

```
# A Python program to demonstrate inheritance
```

```
# Base or Super class. Note object in bracket.  
# (Generally, object is made ancestor of all classes)  
# In Python 3.x "class Person" is  
# equivalent to "class Person(object)"
```

```
class Person(object):
```

```
    # Constructor
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    # To get name
```

```
    def getName(self):
```

```
        return self.name
```

```
    # To check if this person is an employee
```

```
    def isEmployee(self):
```

```
        return False
```

```
# Inherited or Subclass (Note Person in bracket)
```

```
class Employee(Person):
```

```
    # Here we return true
```

```
    def isEmployee(self):
```

```
        return True
```

```
# Driver code
```

```
emp = Person("Geek1") # An Object of Person
```

```
print(emp.getName(), emp.isEmployee())
```

```
emp = Employee("Geek2") # An Object of Employee
```

```
print(emp.getName(), emp.isEmployee())
```

Output:

Geek1 False

Geek2 True

Program:

```
#in file atminfo.py

from AccountOperationsImpl import AccountOperationsImpl

class Account:

    def _init_(self, account_number, name, balance , pin):
        self.account_number = account_number
        self.customer_name = name
        self.balance = balance
        self.pin = pin

    def _str_(self):
        return f'Account_number : {self.account_number}, customer_name : {self.customer_name}, '\
            f'balance : {self.balance}, pin : {self.pin}'

    def _repr_(self):
        return str(self)

a1 = Account(2213821, "Aman", 230000, 14560)
a2= Account(1234567, "Aamir", 23400, 4321)
impl = AccountOperationsImpl()
impl.add_customer(a1)
impl.add_customer(a2)
impl.display_account_info()
print("_____")
impl.deposit_amount(1234567, 20000)
impl.display_account_info()
```

```
print("_____")
impl.withdraw_amount(2213821, 60000)
impl.display_account_info()
print("_____")
impl.change_pin(1234567, 7777)
impl.display_account_info()
print("_____")
#in file accountoperations.py
from abc import ABC, abstractmethod
```

```
class AccountOperations(ABC):
```

```
    @abstractmethod
```

```
    def add_customer(self, cust):
```

```
        pass
```

```
    @abstractmethod
```

```
    def check_balance(self, account_number):
```

```
        pass
```

```
    @abstractmethod
```

```
    def withdraw_amount(self, account_number, amount):
```

```
        pass
```

```
    @abstractmethod
```

```
    def deposit_amount(self, account_number, amount):
```

```
        pass
```

```
    @abstractmethod
```

```
    def change_pin(self, account_number, pin):
```

```
pass
```

```
@abstractmethod
```

```
def display_account_info(self):
```

```
    pass
```

```
#in file accountoperationsimpl.py
```

```
from AccountOperations import AccountOperations
```

```
class AccountOperationsImpl(AccountOperations):
```

```
    list_of_customer = []
```

```
def add_customer(self, cust):
```

```
    self.list_of_customer.append(cust)
```

```
def check_balance(self, account_number):
```

```
    for i in self.list_of_customer:
```

```
        if i.account_number == account_number:
```

```
            print(i)
```

```
def withdraw_amount(self, account_number, amount):
```

```
    for i in self.list_of_customer:
```

```
        if i.account_number == account_number:
```

```
            print(i.balance)
```

```
            i.balance = i.balance-amount
```

```
    print("Amount Withdrawn Successfully ")
```

```
def deposit_amount(self, account_number, amount):
```

```

for i in self.list_of_customer:
    if i.account_number == account_number:
        i.balance = i.balance + amount
print("Amount Deposited Successfully ")

def change_pin(self, account_number, pin):
    for i in self.list_of_customer:
        if i.account_number == account_number:
            i.pin = pin

def display_account_info(self):
    for i in self.list_of_customer:
        print(i)

```

Output:

Account_number : 2213821, customer_name : Aman, balance : 230000, pin : 14560

Account_number : 1234567, customer_name : Aamir, balance : 23400, pin : 4321

Amount Deposited Successfully

Account_number : 2213821, customer_name : Aman, balance : 230000, pin : 14560

Account_number : 1234567, customer_name : Aamir, balance : 43400, pin : 4321

230000

Amount Withdrawn Successfully

Account_number : 2213821, customer_name : Aman, balance : 170000, pin : 14560

Account_number : 1234567, customer_name : Aamir, balance : 43400, pin : 4321

Account_number : 2213821, customer_name : Aman, balance : 170000, pin : 14560

Account_number : 1234567, customer_name : Aamir, balance : 43400, pin : 7777

Conclusion: We have successfully learnt the concept of classes and its inheritance in python and successfully implemented it for atm operations in python.

